



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 03

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

N° de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 12 de Septiembre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Ejercicio 1. Cohete espacial

Descripción

El primer ejercicio consistió en construir un cohete espacial utilizando instancias de diferentes primitivas geométricas que ya estaban implementadas en el programa.

Para construirlo se utilizaron cubos, cilindros, conos, una esfera y pirámides cuadrangulares. Cada una de estas figuras se modificó mediante transformaciones para obtener las diferentes partes del cohete.

La intención principal fue comprobar que una misma primitiva puede reutilizarse varias veces cambiando solamente su posición, escala, rotación y color.

Plataforma

La plataforma del cohete se realizó reutilizando un cubo.

Se aumentó bastante su tamaño en X y Z y se redujo su altura: glm::vec3(6.0f, 0.20f, 6.0f). Esto permite que el cubo se vea como una plataforma horizontal.

```
541 // plataforma del cohete
542 DibujarInstancia(
543     0,
544     glm::vec3(0.0f, -0.10f, 0.0f),
545     glm::vec3(6.0f, 0.20f, 6.0f),
546     glm::vec3(0.0f),
547     glm::vec3(0.10f, 0.10f, 0.10f),
548     uniformModel, uniformColor
549 );
```

Figura 1. Cubo escalado utilizado como plataforma del cohete.

Cuerpo principal

Para el cuerpo principal se utilizó un cilindro.

Su posición fue: glm::vec3(0.0f, 2.40f, 0.0f) y se utilizó una escala: glm::vec3(0.65f, 3.0f, 0.65f). Esto hizo que el cilindro fuera más alto y delgado, permitiendo utilizarlo como cuerpo principal del cohete.

```

551 // cuerpo del cohete, un cilindro gris
552 DibujarInstancia(
553     2,
554     glm::vec3(0.0f, 2.40f, 0.0f),
555     glm::vec3(0.65f, 3.0f, 0.65f),
556     glm::vec3(0.0f),
557     glm::vec3(0.80f, 0.80f, 0.85f),
558     uniformModel, uniformColor
559 );

```

Figura 2. Cilindro utilizado para formar el cuerpo principal.

Punta superior

Para construir la punta del cohete se utilizó un cono de color rojo.

El cono se colocó sobre el cilindro principal y se ajustó su escala para que coincidiera con el tamaño del cuerpo

```

561 // punta roja del cohete
562 DibujarInstancia(
563     3,
564     glm::vec3(0.0f, 4.50f, 0.0f),
565     glm::vec3(0.65f, 1.20f, 0.65f),
566     glm::vec3(0.0f),
567     glm::vec3(0.90f, 0.10f, 0.10f),
568     uniformModel, uniformColor
569 );

```

Figura 3. Cono utilizado para formar la punta del cohete.

Ventana

La ventana del cohete se realizó utilizando una esfera.

Para que no pareciera una esfera completa se redujo principalmente su escala en Z: glm::vec3(0.24f, 0.24f, 0.10f). También se posicionó hacia el frente del cilindro y se utilizó color azul.

```

571 // ventana, esfera aplanada azul
572 DibujarInstancia(
573     5,
574     glm::vec3(0.0f, 2.90f, 0.65f),
575     glm::vec3(0.24f, 0.24f, 0.10f),
576     glm::vec3(0.0f),
577     glm::vec3(0.05f, 0.40f, 1.0f),
578     uniformModel, uniformColor
579 );

```

Figura 4. Esfera escalada utilizada como ventana del cohete.

Propulsores laterales

Los dos propulsores laterales fueron creados reutilizando cilindros.

Uno se colocó en: $x = -1.0f$ y el otro en: $x = 1.0f$. Ambos tienen la misma escala y un color gris.

Sobre cada propulsor también se colocó un cono rojo para completar su forma.

```
581 // propulsor izquierdo, cilindro
582 DibujarInstancia(
583     2,
584     glm::vec3(-1.0f, 1.65f, 0.0f),
585     glm::vec3(0.25f, 2.0f, 0.25f),
586     glm::vec3(0.0f),
587     glm::vec3(0.55f, 0.55f, 0.60f),
588     uniformModel, uniformColor
589 );
590
591 // propulsor derecho, cilindro.
592 DibujarInstancia(
593     2,
594     glm::vec3(1.0f, 1.65f, 0.0f),
595     glm::vec3(0.25f, 2.0f, 0.25f),
596     glm::vec3(0.0f),
597     glm::vec3(0.55f, 0.55f, 0.60f),
598     uniformModel, uniformColor
599 );
600
601 // punta izquierda del propulsor
602 DibujarInstancia(
603     3,
604     glm::vec3(-1.0f, 2.90f, 0.0f),
605     glm::vec3(0.25f, 0.50f, 0.25f),
606     glm::vec3(0.0f),
607     glm::vec3(0.90f, 0.10f, 0.10f),
608     uniformModel, uniformColor
609 );
610
611 // punta derecha del propulsor
612 DibujarInstancia(
613     3,
614     glm::vec3(1.0f, 2.90f, 0.0f),
615     glm::vec3(0.25f, 0.50f, 0.25f),
616     glm::vec3(0.0f),
617     glm::vec3(0.90f, 0.10f, 0.10f),
618     uniformModel, uniformColor
619 );
```

Figura 5. Cilindros y conos utilizados en los propulsores laterales.

Aletas

Las aletas fueron realizadas mediante pirámides cuadrangulares.

Se colocaron cuatro:

- Izquierda.
- Derecha.
- Delantera.
- Trasera.

En las aletas laterales se utilizó: `glm::vec3(0.75f, 1.10f, 0.20f)`, mientras que en las aletas frontal y trasera se intercambiaron las escalas para cambiar su orientación: `glm::vec3(0.20f, 1.10f, 0.75f)`.

También fue necesario ajustar su posición vertical para evitar que quedaran demasiado abajo y se encimaran con los propulsores.

```
621 // aleta izquierda
622 DibujarInstancia(
623     4,
624     glm::vec3(-0.55f, 1.50f, 0.0f),
625     glm::vec3(0.75f, 1.10f, 0.20f),
626     glm::vec3(0.0f),
627     glm::vec3(0.90f, 0.10f, 0.10f),
628     uniformModel, uniformColor
629 );
630
631
632 // aleta derecha
633 DibujarInstancia(
634     4,
635     glm::vec3(0.55f, 1.50f, 0.0f),
636     glm::vec3(0.75f, 1.10f, 0.20f),
637     glm::vec3(0.0f),
638     glm::vec3(0.90f, 0.10f, 0.10f),
639     uniformModel, uniformColor
640 );
641
642
643 // aleta delantera
644 DibujarInstancia(
645     4,
646     glm::vec3(0.0f, 1.50f, 0.55f),
647     glm::vec3(0.20f, 1.10f, 0.75f),
648     glm::vec3(0.0f),
649     glm::vec3(0.90f, 0.10f, 0.10f),
650     uniformModel, uniformColor
651 );
652
```

Figura 6. Pirámides cuadrangulares utilizadas como aletas.

Fuego de los motores

Para representar el fuego se utilizaron conos invertidos.

La inversión se consiguió utilizando: `glm::vec3(180.0f, 0.0f, 0.0f)`. Se colocó una llama debajo del cuerpo principal y otra debajo de cada propulsor.

```
664 // fuego central, cono invertido
665 DibujarInstancia(
666     3,
667     glm::vec3(0.0f, 0.50f, 0.0f),
668     glm::vec3(0.40f, 0.80f, 0.40f),
669     glm::vec3(180.0f, 0.0f, 0.0f),
670     glm::vec3(1.0f, 0.45f, 0.0f),
671     uniformModel, uniformColor
672 );
673
674 // fuego del propulsor izquierdo
675 DibujarInstancia(
676     3,
677     glm::vec3(-1.0f, 0.40f, 0.0f),
678     glm::vec3(0.18f, 0.50f, 0.18f),
679     glm::vec3(180.0f, 0.0f, 0.0f),
680     glm::vec3(1.0f, 0.70f, 0.0f),
681     uniformModel, uniformColor
682 );
683
684 // fuego del propulsor derecho
685 DibujarInstancia(
686     3,
687     glm::vec3(1.0f, 0.40f, 0.0f),
688     glm::vec3(0.18f, 0.50f, 0.18f),
689     glm::vec3(180.0f, 0.0f, 0.0f),
690     glm::vec3(1.0f, 0.70f, 0.0f),
691     uniformModel, uniformColor
692 );
```

Figura 7. Conos invertidos utilizados para representar el fuego.

Resultado

El resultado fue un cohete tridimensional construido completamente mediante instancias de primitivas geométricas.

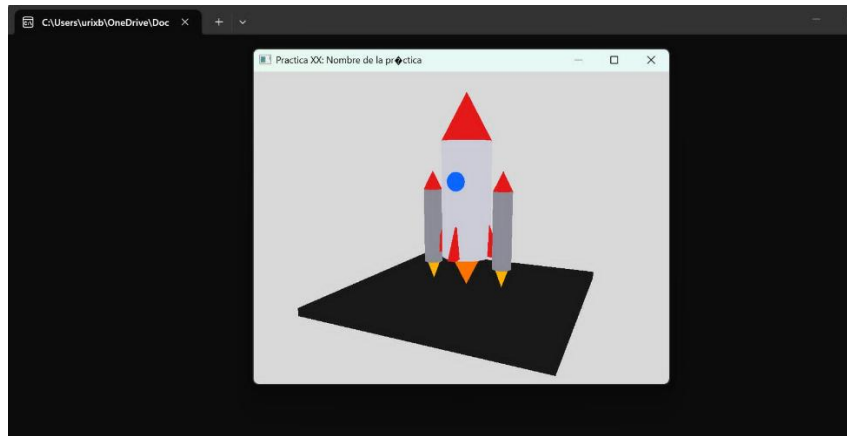


Figura 8. Resultado final del cohete espacial.

Ejercicio 2. Unión de 8 pirámides

Descripción

El segundo ejercicio consistió en construir una figura tridimensional formada por ocho pirámides.

Las pirámides se distribuyeron en cuatro posiciones principales: arriba, derecha, abajo, izquierda. En cada posición se colocaron dos pirámides, una orientada hacia afuera y otra hacia el centro, dando un total de ocho.

Creación de la pirámide por caras

Para este ejercicio se creó una pirámide formada por diferentes Mesh.

Se utilizaron cuatro vértices para la base y uno adicional para la punta.

Después se creó cada cara por separado: frente, derecha, atrás, izquierda, base

```

443 void CrearPiramidePorCaras()
444 {
445     // cuatro esquinas de la base y punta.
446     GLfloat vertices[] = {
447         -0.65f, 0.0f, 0.65f, // 0
448         0.65f, 0.0f, 0.65f, // 1
449         0.65f, 0.0f, -0.65f, // 2
450         -0.65f, 0.0f, -0.65f, // 3
451         0.0f, 1.4f, 0.0f // 4: punta
452     };
453
454     unsigned int frente[] = { 0, 1, 4 };
455     unsigned int derecha[] = { 1, 2, 4 };
456     unsigned int atras[] = { 2, 3, 4 };
457     unsigned int izquierda[] = { 3, 0, 4 };
458     unsigned int base[] = { 0, 3, 2, 0, 2, 1 };
459
460     Mesh* caraRoja = new Mesh();
461     caraRoja->CreateMesh(vertices, frente, 15, 3);
462     carasPiramide.push_back(caraRoja);
463
464     Mesh* caraVerde = new Mesh();
465     caraVerde->CreateMesh(vertices, derecha, 15, 3);
466     carasPiramide.push_back(caraVerde);
467
468     Mesh* caraAmarilla = new Mesh();
469     caraAmarilla->CreateMesh(vertices, atras, 15, 3);
470     carasPiramide.push_back(caraAmarilla);
471
472     Mesh* caraMagenta = new Mesh();
473     caraMagenta->CreateMesh(vertices, izquierda, 15, 3);
474     carasPiramide.push_back(caraMagenta);
475
476     Mesh* caraAzul = new Mesh();
477     caraAzul->CreateMesh(vertices, base, 15, 6);
478     carasPiramide.push_back(caraAzul);
479 }

```

Figura 9. Creación de las caras independientes de la pirámide.

Colores

La función: DibujarPiramideColores() asigna un color diferente a cada cara. Los colores utilizados fueron rojo, verde, amarillo, magenta y azul.

```

481 void DibujarPiramideColores(
482     glm::mat4 model,
483     GLuint uniformModel,
484     GLuint uniformColor)
485 {
486     glm::vec3 colores[] = {
487         glm::vec3(1.0f, 0.0f, 0.0f), // rojo
488         glm::vec3(0.0f, 1.0f, 0.0f), // verde
489         glm::vec3(1.0f, 1.0f, 0.0f), // amarillo
490         glm::vec3(1.0f, 0.0f, 1.0f), // magenta
491         glm::vec3(0.0f, 0.0f, 1.0f) // azul
492     };
493
494     glUniformMatrix4fv(
495         uniformModel,
496         1,
497         GL_FALSE,
498         glm::value_ptr(model)
499     );
500
501     for (int cara = 0; cara < 5; cara++)
502     {
503         glUniform3fv(
504             uniformColor,
505             1,
506             glm::value_ptr(colores[cara])
507         );
508
509         carasPiramide[cara]->RenderMesh();
510     }
511 }

```

Figura 10. Asignación de colores a las caras de la pirámide.

Posiciones y rotaciones

Para colocar las ocho pirámides se utilizó el arreglo: posiciones[] Se definieron dos posiciones iguales para cada dirección debido a que en cada punto se colocaron dos pirámides.

También se utilizaron: girosZ[] y girosY[] para controlar la orientación de cada figura. Finalmente se utilizó: for (int i = 0; i < 8; i++) para generar las ocho instancias.

```

642 // gira cada piramide sobre su eje para mostrar
643 float girosY[] = {
644     0.0f, 90.0f,
645     180.0f, -90.0f,
646     90.0f, 0.0f,
647     -90.0f, 180.0f
648 };
649
650 for (int i = 0; i < 8; i++)
651 {
652     glm::mat4 model = glm::translate(
653         conjunto,
654         posiciones[i]
655     );
656
657     model = glm::rotate(
658         model,
659         glm::radians(girosZ[i]),
660         glm::vec3(0.0f, 0.0f, 1.0f)
661     );
662
663     model = glm::rotate(
664         model,
665         glm::radians(girosY[i]),
666         glm::vec3(0.0f, 1.0f, 0.0f)
667     );
668
669     model = glm::scale(
670         model,
671         glm::vec3(1.0f, 0.65f / 1.4f, 1.0f)
672     );
673
674     DibujarPiramideColores(
675         model,
676         uniformModel,
677         uniformColor
678     );
679 }

```

Figura 11. Posiciones y rotaciones utilizadas para acomodar las ocho pirámides.

Rotación de la figura completa

Para controlar toda la estructura se utilizó una matriz llamada: conjunto. Sobre esta matriz se aplican las rotaciones de los ejes X, Y y Z.

Esto permite girar la unión completa de pirámides sin tener que modificar cada objeto individualmente.

```

606 // ejercicio 2 practica 3 figura
607 // transformacion de toda la figura.
608 glm::mat4 conjunto = glm::mat4(1.0f);
609
610 // permite girar la union
611 conjunto = glm::rotate(conjunto,glm::radians(mainWindow.getrotax()),glm::vec3(1.0f, 0.0f, 0.0f));
612
613 conjunto = glm::rotate(conjunto,glm::radians(mainWindow.getrotay()),glm::vec3(0.0f, 1.0f, 0.0f));
614
615 conjunto = glm::rotate(conjunto,glm::radians(mainWindow.getrotaz()),glm::vec3(0.0f, 0.0f, 1.0f));
616

```

Figura 12. Transformación utilizada para rotar toda la unión.

Resultado

El resultado fue una estructura tridimensional formada por ocho pirámides orientadas en diferentes direcciones y con varias caras de colores.

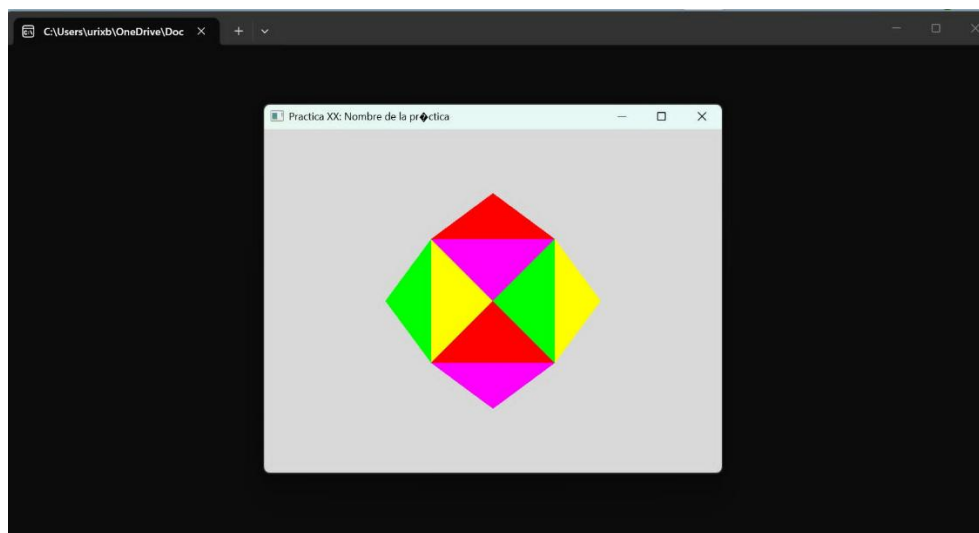


Figura 13. Resultado final de la unión de ocho pirámides.

Problemas presentados

En el ejercicio del cohete, uno de los principales problemas que tuve fue acomodar correctamente todas las primitivas para que el modelo mantuviera una forma reconocible y proporcionada. Fue necesario que realizara varios ajustes en las posiciones y escalas del cuerpo, propulsores, fuego y principalmente de las aletas.

Las aletas inicialmente estaban demasiado abajo y llegaban a encimarse con los propulsores, por lo que modifiqué su altura y tamaño hasta conseguir una posición más adecuada.

En el ejercicio de las ocho pirámides, una dificultad importante fue interpretar la imagen de referencia. La imagen no me permitía apreciar claramente cómo debía estar formada realmente la estructura tridimensional ni hacia dónde debía apuntar cada pirámide.

Por esta razón fue necesario que realizara varias pruebas con posiciones y rotaciones para deducir una distribución que se acercara lo más posible a lo solicitado. Esta parte hizo que el ejercicio fuera más complicado porque primero fue necesario interpretar la estructura y después programarla.

También fue necesario que separa la pirámide en diferentes caras para poder utilizar distintos colores dentro de una misma figura.

Conclusión:

- a) La práctica tuvo una complejidad mayor que las anteriores porque se combinaron distintas primitivas y transformaciones dentro de una misma escena. El cohete requirió principalmente trabajo de posicionamiento y escalado, ya que cada figura debía combinarse con las demás para formar un solo modelo. La unión de ocho pirámides fue más complicada en la parte de las rotaciones, debido a que cada una debía tener una orientación específica para mantener la forma del conjunto. En ambos ejercicios, la función `DibujarInstancia()` me facilitó bastante el trabajo al permitir reutilizar las mismas figuras con diferentes valores de posición, escala, rotación y color.
- b) Realizar estos ejercicios me permitieron entender mejor cómo varias figuras geométricas simples pueden combinarse para formar objetos más elaborados. El cohete fue útil para trabajar con diferentes tipos de primitivas dentro de un mismo modelo, mientras que la unión de las pirámides ayudó a practicar de una forma más directa el uso de rotaciones y transformaciones.
- c) Conclusión: Al finalizar esta práctica se concluye que mediante la instanciación y transformación de primitivas geométricas es posible construir modelos tridimensionales más complejos sin necesidad de crear una geometría completamente nueva para cada elemento. En el ejercicio del cohete se utilizaron cubos, cilindros, conos, esfera y pirámides para formar un solo objeto, mientras que en el segundo ejercicio se trabajó principalmente con posiciones y rotaciones para construir una unión de ocho pirámides, esto ayudó a reforzar el uso de transformaciones, instancias, colores, cámara sintética y proyección en perspectiva dentro de una escena 3D.

Bibliografía

- Kanetrix GC. (2011). *Tutorial 2 OpenGL: Transformación de vistas y cámaras*. Recuperado el 13 de septiembre de 2026, de <https://kanetrixgc.wordpress.com/2011/05/06/tutorial-2-opengl-transformacion-de-vistas-y-camaras/>
- Departamento de Computación, CINVESTAV. (s. f.). *Curso de graficación: Transformaciones geométricas, visión 3D y modelado de sólidos*.

Recuperado el 13 de septiembre de 2026, de
<https://wwwcs.cs.cinvestav.mx/cursos/Graficacion.html>